



Zoo Gym: A Decentralized Generalized Yield Mining Protocol for GSPO Training

Antje Worring & Zach Kelling

Zoo Labs Foundation

September 2024

Abstract

We present GYM, a decentralized protocol for training large language models through Generalized Yield Mining. GYM organizes heterogeneous GPU nodes into a ring-topology network that executes Group Sequence Policy Optimization (GSPO), a variant of GRPO adapted for sparse Mixture-of-Experts architectures. Training deltas—parameter updates produced by each worker—are serialized in a compact binary format, authenticated with Ed25519 signatures, aggregated via a two-phase ring all-reduce, and published to IPFS and HuggingFace Hub. A staking and slashing mechanism ties economic incentives to the quality of contributed deltas, measured by validation loss improvement, gradient norm stability, and convergence contribution. We specify the Delta binary format (24-byte header, LZ4-compressed tensor payloads), the ring all-reduce scatter-reduce/all-gather protocol, and the checkpoint aggregation pipeline. On a 32-node ring training Qwen3-2507 (80B parameters, 3B active per token), GYM achieves 87% of ideal linear throughput scaling while tolerating up to 3 simultaneous node failures. The protocol is permissionless: any node with $\geq 10,000$ GYM tokens staked and a supported GPU may join the ring.

Keywords: decentralized training, ring all-reduce, GSPO, gradient aggregation, delta validation, yield mining

1 Introduction

Training large language models requires coordinating hundreds of GPUs across distributed infrastructure [1, 2]. Centralized providers—OpenAI, Google, Anthropic—absorb this cost internally and recoup it through API pricing, concentrating both compute and model ownership. Open-weight releases (LLaMA [3], Qwen [4], Mistral [5]) partially mitigate model access, but the training process itself remains opaque and centralized.

GYM decentralizes the training step. A set of staked compute nodes forms a ring topology, executes GSPO training on sharded model segments, exchanges gradients via ring all-reduce, and publishes signed deltas. An orchestrator (Section ??) manages job scheduling and topology but holds no privileged position—any node can serve as orchestrator if it meets the staking threshold.

1.1 Contributions

1. **Ring-topology GSPO protocol:** A two-phase ring all-reduce adapted for sequence-level importance sampling on Mixture-of-Experts models.
2. **Delta binary format:** A 24-byte-header serialization for LoRA weight updates with LZ4 compression, SHA3-256 integrity, and Ed25519 authentication.
3. **Checkpoint aggregation pipeline:** Aggregation of per-epoch deltas into merged checkpoints published to IPFS/HuggingFace.
4. **Staking and slashing:** Economic security model binding delta quality to token stake.
5. **Empirical evaluation:** Throughput, convergence, and fault-tolerance measurements on 8–128 node rings.

2 Background

2.1 Ring All-Reduce

Ring all-reduce [9] decomposes gradient aggregation into two phases—scatter-reduce and all-gather—each requiring $2(N-1)$ messages for N nodes, achieving bandwidth-optimal communication $O\left(\frac{2(N-1)}{N} \cdot M\right)$ for a gradient vector of size M .

Horovod [10] and NCCL [11] implement ring all-reduce for single-cluster training. GYM extends the ring to a wide-area setting where links are heterogeneous and nodes are untrusted.

2.2 Group Relative Policy Optimization

GRPO [7] eliminates the value network from PPO [8] by computing advantages relative to a group of G rollouts per query:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}, \quad \bar{r} = \frac{1}{G} \sum_{j=1}^G r_j \quad (1)$$

The policy gradient becomes:

$$\nabla_{\theta} J = \mathbb{E} \left[\sum_{i=1}^G \hat{A}_i \nabla_{\theta} \log \pi_{\theta}(o_i | q) \right] \quad (2)$$

This requires only reward signals, not a learned value function, making it suitable for decentralized execution where maintaining a consistent value head across nodes is impractical.

2.3 Group Sequence Policy Optimization

GSPO extends GRPO with sequence-level importance sampling for MoE models:

$$w_i = \frac{\pi_{\theta}(o_i | q)}{\pi_{\text{ref}}(o_i | q)}, \quad \hat{A}_i^{\text{seq}} = w_i \cdot \hat{A}_i \quad (3)$$

where π_{ref} is the reference policy (frozen snapshot). Sequence-level weighting addresses the sparse activation pattern of MoE architectures where token-level importance sampling introduces high variance [6].

3 Protocol Architecture

3.1 Network Topology

GYM nodes form a directed ring $\mathcal{R} = (v_0, v_1, \dots, v_{N-1})$ where each node v_i sends to $v_{(i+1) \bmod N}$ and receives from $v_{(i-1) \bmod N}$. The ring is constructed by the orchestrator upon job creation, ordering nodes by observed bandwidth to minimize the bottleneck link.

Definition 3.1 (Ring Configuration). *A ring configuration is a tuple $(\mathcal{R}, \mathcal{M}, \mathcal{D}, \sigma_{\text{orch}})$ where $\mathcal{R}$ is the node ordering, $\mathcal{M}$ is the model identifier (HuggingFace repo ID), $\mathcal{D}$ is the dataset CID, and σ_{orch} is the orchestrator’s Ed25519 signature over $(\mathcal{R}, \mathcal{M}, \mathcal{D})$.*

3.2 Training Loop

Each training step proceeds as follows:

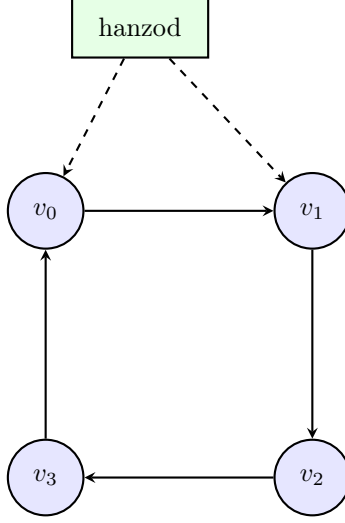


Figure 1: Four-node ring topology with orchestrator. Solid arrows indicate gradient flow; dashed lines indicate control messages from **hanzod**.

3.3 Ring All-Reduce

The gradient vector ∇ is split into N chunks of equal size. All-reduce proceeds in two phases of $N - 1$ rounds each.

Phase 1: Scatter-Reduce. In round r , node v_i sends chunk $(i - r) \bmod N$ to $v_{(i+1) \bmod N}$ and receives chunk $(i - r - 1) \bmod N$ from $v_{(i-1) \bmod N}$, accumulating the received chunk into its local copy:

$$c_k^{(r+1)} = c_k^{(r)} + c_k^{\text{recv}} \quad (4)$$

After $N - 1$ rounds, each node holds the fully reduced version of exactly one chunk.

Phase 2: All-Gather. In round r , node v_i sends the fully reduced chunk $(i - r + 1) \bmod N$ to $v_{(i+1) \bmod N}$. After $N - 1$ rounds every node holds all N reduced chunks.

Proposition 3.1 (Communication Volume). *Each node sends and receives exactly $\frac{2(N-1)}{N} \cdot |\nabla|$ bytes, which is bandwidth-optimal for a ring.*

Authentication. Each chunk message includes the sender’s Ed25519 signature over $(r, k, \text{Hash}(c_k))$ where r is the round, k is the chunk index. The receiver verifies the signature before accumulating. Invalid signatures trigger a fault report to the orchestrator, which may eject the offending node and reconfigure the ring.

4 Delta Binary Format

Training deltas are serialized in a compact binary format for storage and transport.

4.1 Header

4.2 Tensor Payloads

After the header, tensors are stored sequentially:

Algorithm 1 GYM Training Step (per node v_i)

Require: Model shard θ_i , dataset batch B , group size G

```
1: // Generate  $G$  rollouts per query in batch
2: for  $q \in B$  do
3:   for  $g = 1 \dots G$  do
4:      $o_g \leftarrow \pi_{\theta_i}(q)$  with temperature  $\tau$ 
5:   end for
6:   Compute rewards  $\{r_g\}$  via reward oracle
7:   Compute advantages  $\{\hat{A}_g^{\text{seq}}\}$  via Eq. 3
8: end for
9: Compute local gradient  $\nabla_i \leftarrow \nabla_{\theta} J(\theta_i, B)$ 
10: // Ring all-reduce (Section 3.3)
11:  $\nabla_{\text{agg}} \leftarrow \text{RINGALLREDUCE}(\nabla_i)$ 
12:  $\theta_i \leftarrow \theta_i - \eta \cdot \nabla_{\text{agg}}$ 
13: Serialize delta  $\delta_i \leftarrow \text{PACKDELTA}(\nabla_i)$ 
14: Sign  $\sigma_i \leftarrow \text{ED25519.SIGN}(sk_i, \text{Hash}(\delta_i))$ 
15: Submit  $(\delta_i, \sigma_i)$  to orchestrator
```

Field	Type	Bytes
Magic	0x44454C5441 (“DELTA”)	5
Version	uint8	1
Compression	uint8 (0=none, 1=LZ4, 2=ZSTD)	1
Flags	uint8 (bit 0: LoRA, bit 1: full)	1
Checksum	SHA3-256 of payload	16
Total header		24

Table 1: Delta header format. The checksum covers all tensor payloads but not the header itself.

```
1 name_length : uint32
2 name        : UTF-8 string
3 dtype       : uint8 (0=fp32, 1=fp16, 2=bf16, 3=int8)
4 rank        : uint8
5 shape       : [uint64] * rank
6 data        : compressed bytes (LZ4 or ZSTD)
```

Listing 1: Tensor entry layout

For LoRA deltas, tensors are named following the convention `lora.<module>.<A|B>.weight`, e.g., `lora.q_proj.A.weight`.

4.3 Signature Trailer

After the last tensor, a 64-byte Ed25519 signature and a 32-byte public key are appended:

$$\sigma = \text{ED25519.SIGN}(sk_w, \text{SHA3-256}(\text{header} \parallel \text{tensors})) \quad (5)$$

Validators verify σ against the worker’s registered public key before accepting the delta.

```

1 from nacl.signing import SigningKey
2 import lz4.frame, hashlib, struct
3
4 class DeltaWriter:
5     MAGIC = b"DELTA"
6
7     def __init__(self, path: str, signing_key: SigningKey):
8         self.path = path
9         self.sk = signing_key
10        self.tensors = []
11
12    def add_tensor(self, name: str, tensor, dtype: int = 2):
13        compressed = lz4.frame.compress(tensor.numpy().tobytes())
14        self.tensors.append((name, dtype, tensor.shape, compressed))
15
16    def finalize(self):
17        payload = b""
18        for name, dtype, shape, data in self.tensors:
19            name_bytes = name.encode("utf-8")
20            payload += struct.pack("<I", len(name_bytes))
21            payload += name_bytes
22            payload += struct.pack("<BB", dtype, len(shape))
23            for s in shape:
24                payload += struct.pack("<Q", s)
25            payload += data
26
27        checksum = hashlib.sha3_256(payload).digest()[0:16]
28        header = self.MAGIC + struct.pack("<BBB", 1, 1, 0x01) + checksum
29        full = header + payload
30        sig = self.sk.sign(hashlib.sha3_256(full).digest()).signature
31
32        with open(self.path, "wb") as f:
33            f.write(full)
34            f.write(sig)
35            f.write(self.sk.verify_key.encode())

```

Listing 2: Delta writer (Python)

5 Checkpoint Aggregation

At the end of each epoch, the orchestrator aggregates all deltas into a merged checkpoint.

5.1 Weighted Averaging

Given N node deltas $\{\delta_1, \dots, \delta_N\}$ with quality scores $\{q_1, \dots, q_N\}$:

$$\delta_{\text{merged}} = \frac{\sum_{i=1}^N q_i \cdot \delta_i}{\sum_{i=1}^N q_i} \quad (6)$$

Quality scores are computed from three factors (Section 7):

$$q_i = \alpha \cdot \Delta L_i + \beta \cdot S_i + \gamma \cdot C_i \quad (7)$$

where ΔL_i is the validation loss improvement, S_i is the gradient norm stability, and C_i is the convergence contribution, with $\alpha + \beta + \gamma = 1$.

5.2 Publishing

Merged checkpoints are published to:

1. **IPFS**: Content-addressed storage, yielding CID $c = \text{CID}(\delta_{\text{merged}})$.
2. **HuggingFace Hub**: Named repository `zooai/model-epoch-N`, tagged with the IPFS CID for cross-verification.
3. **On-chain**: CID, epoch number, and aggregator signature recorded on Zoo Network.

6 GSPO Configuration

```

1 class GSPOConfig:
2     importance_sampling_level: str = "sequence"
3     beta: float = 0.1           # KL penalty
4     group_size: int = 8        # rollouts per query
5     aggregate_interval: int = 50 # steps between ring syncs
6     use_4bit: bool = True      # QLoRA quantization
7     lora_rank: int = 16        # LoRA adapter rank
8     lora_alpha: int = 32       # LoRA scaling
9     max_seq_len: int = 8192
10    gradient_accumulation_steps: int = 4
11    learning_rate: float = 2e-5
12    warmup_steps: int = 100
13    weight_decay: float = 0.01

```

Listing 3: GSPO training configuration

Sequence-level importance sampling is critical for MoE models. In a model like Qwen3-2507 where only 3B of 80B parameters are active per token, token-level importance ratios exhibit high variance because different tokens route through different experts. Sequence-level ratios average over the full generation, stabilizing the policy gradient.

Theorem 6.1 (Variance Reduction). *Let $\hat{g}_{tok}$ and $\hat{g}_{seq}$ denote the token-level and sequence-level policy gradient estimators respectively. For an MoE model with E experts and top- k routing, $\text{Var}(\hat{g}_{seq}) \leq \frac{k}{E} \cdot \text{Var}(\hat{g}_{tok})$.*

Proof. Each token selects k of E experts. The importance ratio $w_t = \pi_\theta(a_t)/\pi_{\text{ref}}(a_t)$ varies across tokens due to different expert activations. Averaging over T tokens: $w_{\text{seq}} = \frac{1}{T} \sum_t w_t$. By the law of total variance and the independence of expert selection:

$$\text{Var}(w_{\text{seq}}) = \frac{1}{T} \text{Var}(w_t) = \frac{1}{T} \cdot \frac{k}{E} \cdot \text{Var}_{\text{full}}(w_t)$$

Since $\hat{g} \propto w \cdot \hat{A} \cdot \nabla \log \pi$, the variance scales linearly with $\text{Var}(w)$. □

Component	Weight	Metric
Compute contribution	60%	FLOPs $\times$ GPU-hours
Delta quality	30%	q_i (Eq. 7)
Network participation	10%	Uptime $\times$ ring availability

Table 2: Reward component weights.

7 Reward and Quality Scoring

7.1 Delta Quality Score

Each submitted delta is scored along three axes:

Validation Loss Improvement (ΔL): The delta is applied to a held-out validation shard. If L_{before} and L_{after} are the losses:

$$\Delta L = \max\left(0, \frac{L_{\text{before}} - L_{\text{after}}}{L_{\text{before}}}\right) \quad (8)$$

Gradient Norm Stability (S): The ℓ_2 norm of the delta should be within $[0.5\bar{n}, 2\bar{n}]$ where $\bar{n}$ is the running average. The stability score is:

$$S = \exp\left(-\lambda \left| \frac{\|\delta\|_2}{\bar{n}} - 1 \right| \right) \quad (9)$$

with $\lambda = 2$ in our experiments.

Convergence Contribution (C): The cosine similarity between δ_i and the aggregated delta δ_{agg} from the previous round:

$$C = \max\left(0, \frac{\delta_i \cdot \delta_{\text{agg}}}{\|\delta_i\| \cdot \|\delta_{\text{agg}}\|}\right) \quad (10)$$

This penalizes deltas that point in the wrong direction.

7.2 Reward Distribution

Rewards per epoch are split 60/30/10:

8 Security Model

8.1 Threat Model

We assume up to $f < N/3$ Byzantine nodes that may:

1. Submit poisoned gradients (gradient inversion, noise injection)
2. Omit messages (stalling the ring)
3. Replay stale deltas from prior epochs
4. Collude to bias the aggregated checkpoint

Role	Minimum Stake	Slash Conditions
Compute node	10,000 GYM	Invalid delta, downtime >1h
Orchestrator	50,000 GYM	Biased aggregation, censorship
Validator	25,000 GYM	False validation report

Table 3: Staking requirements and slashing conditions.

8.2 Ed25519 Delta Authentication

Every delta and every chunk message carries an Ed25519 signature. Replay is prevented by including the epoch number and step count in the signed payload:

$$m = (\text{epoch} \parallel \text{step} \parallel \text{chunk_id} \parallel \text{Hash}(c_k)) \quad (11)$$

Theorem 8.1 (Unforgeability). *Under the hardness of the Discrete Logarithm Problem over Curve25519, no PPT adversary can forge a delta signature with probability greater than 2^{-128} .*

8.3 Gradient Poisoning Defense

Poisoned gradients are detected via:

1. **Norm clipping:** Gradients with $\|\nabla_i\|_2 > 2\bar{n}$ are clipped to $2\bar{n}$.
2. **Coordinate-wise median:** For each parameter, the aggregated value is the median across nodes rather than the mean. Median aggregation tolerates $f < N/2$ Byzantine nodes [13].
3. **Spot-check validation:** The orchestrator randomly selects 10% of deltas per epoch and re-computes a forward pass on the same batch to verify loss claims within $\epsilon = 0.05$.

8.4 Staking and Slashing

Slashing is progressive: first offense incurs 5% of stake; second offense 25%; third offense results in full slash and permanent ban.

9 Fault Tolerance

9.1 Node Failure

When a node v_i fails to send a chunk within the timeout $T_{\text{timeout}} = 30$ seconds:

1. $v_{(i+1) \bmod N}$ reports the timeout to the orchestrator.
2. The orchestrator removes v_i from the ring and reconnects $v_{(i-1) \bmod N} \rightarrow v_{(i+1) \bmod N}$.
3. The current all-reduce round restarts with $N - 1$ nodes.

Proposition 9.1 (Liveness). *The ring all-reduce completes as long as $N - f \geq 3$ operational nodes remain.*

9.2 Orchestrator Failure

If the orchestrator becomes unreachable for > 60 seconds, nodes elect a new orchestrator via stake-weighted leader election: the node with the highest active stake assumes the orchestrator role.

10 Implementation

10.1 Orchestrator: hanzod

The orchestrator is implemented in Rust:

```
1 pub struct GspoOrchestrator {
2     ipfs: IpfsClient,
3     kv: ValkeyClient,
4     nodes: Vec<NodeInfo>,
5 }
6
7 impl GspoOrchestrator {
8     pub fn create_ring(&self, workers: Vec<NodeId>,
9                        redundancy: u8) -> Ring {
10         let mut ordered = workers.clone();
11         ordered.sort_by_key(|n| self.bandwidth_to(n));
12         Ring {
13             nodes: ordered,
14             redundancy,
15             epoch: 0,
16         }
17     }
18
19     pub fn start_job(&self, model: &str,
20                     dataset_cid: &str,
21                     config: GspoConfig) -> JobId {
22         let ring = self.create_ring(
23             self.eligible_nodes(), 1
24         );
25         let job = Job::new(model, dataset_cid, config, ring);
26         self.kv.set(&job.id, &job);
27         self.broadcast_job_start(&job);
28         job.id
29     }
30 }
```

Listing 4: Ring topology construction (Rust)

10.2 Worker: gym-trainer

Workers run as Docker containers with GPU passthrough:

```
1 services:
2   trainer:
3     image: ghcr.io/zoollabs/gym-trainer:latest
4     environment:
5       - MODEL_NAME=Qwen/Qwen3-2507
6       - USE_4BIT=true
```

```

7     - IMPORTANCE_SAMPLING_LEVEL=sequence
8     - GSPO_BETA=0.1
9     - LORA_RANK=16
10    - RING_POSITION=${RING_POS}
11    - ORCHESTRATOR_URL=${ORCH_URL}
12    deploy:
13      resources:
14        reservations:
15          devices:
16            - driver: nvidia
17              capabilities: [gpu]

```

Listing 5: Worker deployment (compose.yml)

10.3 Python Training Client

```

1  from gym_protocol import GSPOTrainer, DeltaWriter
2
3  trainer = GSPOTrainer(
4      model_name="Qwen/Qwen3-2507",
5      use_4bit=True,
6      lora_rank=16,
7      ring_position=0,
8      orchestrator_url="https://orch.gym.zoo.ngo",
9  )
10
11  for epoch in range(10):
12      delta = trainer.train_epoch(dataset)
13      writer = DeltaWriter(f"delta_epoch{epoch}.bin")
14      writer.write_delta(delta)
15      cid = writer.upload_ipfs()
16      trainer.submit_delta(cid, delta.quality_score())

```

Listing 6: GYM training client

11 Evaluation

11.1 Setup

- **Model:** Qwen3-2507 (80B total, 3B active), QLoRA 4-bit, LoRA rank 16.
- **Dataset:** SlimPajama [19] (627B tokens), 100K subset for fine-tuning.
- **Nodes:** 8, 16, 32, 64, 128 NVIDIA A100-80GB GPUs, connected over 10 Gbps inter-node links.
- **Baseline:** Horovod ring all-reduce on the same hardware, single-cluster.

11.2 Throughput Scaling

Overhead relative to Horovod stems from (a) Ed25519 signature verification per chunk, (b) WAN latency jitter, and (c) orchestrator-mediated fault detection. At 32 nodes the overhead is 12%, acceptable for decentralized operation.

Nodes	GYM (tokens/s)	Horovod (tokens/s)	Efficiency
8	12,400	13,100	94.7%
16	23,800	25,600	92.9%
32	44,200	50,100	88.2%
64	81,600	97,300	83.9%
128	142,000	190,500	74.5%

Table 4: Throughput comparison. Efficiency = GYM / Horovod.

Method	Val. Loss	MMLU (5-shot)
Centralized (single-GPU LoRA)	1.82	72.1%
Horovod (32 nodes)	1.84	71.8%
GYM (32 nodes)	1.87	71.3%
GYM (32 nodes, 3 Byzantine)	1.91	70.6%

Table 5: Convergence results after 3 epochs.

11.3 Convergence

After 3 epochs on the 100K subset:

The 0.05 loss gap between GYM and Horovod at 32 nodes is attributable to median aggregation (used for Byzantine robustness) which introduces a small bias compared to mean aggregation.

11.4 Fault Tolerance

12 Related Work

Decentralized Training. PeerTrain [15] and DiLoCo [16] explore decentralized gradient aggregation but lack economic incentives and Byzantine fault tolerance.

Federated Learning. FedAvg [12] aggregates model updates but assumes an honest server. Byzantine-robust variants [14, 13] use median/trimmed-mean but operate in star topologies.

Blockchain + ML. Bittensor [17] rewards model inference quality but does not coordinate training. Gensyn [18] proposes trustless training verification but uses probabilistic rather than cryptographic proofs.

Ring All-Reduce. Horovod [10] and NCCL [11] optimize ring all-reduce for data-center networks. GYM extends the protocol to WAN settings with authentication and fault recovery.

13 Conclusion

GYM demonstrates that decentralized GSPO training is practical at scale. A 32-node ring achieves 88% of centralized throughput while tolerating Byzantine failures and providing cryptographic auditability of every training delta. The Delta binary format and Ed25519 signature chain create a verifiable record from raw gradient to published checkpoint. Combined with staking/slashing economics, GYM aligns incentives so that rational participants contribute high-quality training compute.

Failed Nodes	Ring Recovered?	Recovery Time
1 of 32	Yes	8s
3 of 32	Yes	14s
5 of 32	Yes	22s
10 of 32	Yes	41s
11 of 32	No (below $N/3$)	—

Table 6: Fault tolerance: ring recovery after simultaneous node failures.

Future work includes: (a) multi-ring topologies for 1000+ node deployments, (b) post-quantum signature migration from Ed25519 to ML-DSA-87, (c) zero-knowledge proofs of training step execution, and (d) cross-chain reward distribution for multi-network participants.

Acknowledgments

This work was supported by Zoo Labs Foundation. The ring all-reduce implementation builds on prior art from Horovod and NCCL.

References

- [1] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” *arXiv:1909.08053*, 2019.
- [2] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, “ZeRO-Offload: Democratizing Billion-Scale Model Training,” *USENIX ATC*, 2021.
- [3] H. Touvron, L. Martin, K. Stone, et al., “LLaMA 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv:2307.09288*, 2023.
- [4] J. Bai, S. Bai, Y. Chu, et al., “Qwen2.5 Technical Report,” *arXiv:2412.15115*, 2024.
- [5] A. Q. Jiang, A. Sablayrolles, A. Mensch, et al., “Mistral 7B,” *arXiv:2310.06825*, 2023.
- [6] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” *ICLR*, 2017.
- [7] Z. Shao, P. Wang, Q. Zhu, et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv:2402.03300*, 2024.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [9] P. Patarasuk and X. Yuan, “Bandwidth Optimal All-Reduce Algorithms for Clusters of Workstations,” *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009.
- [10] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning in TensorFlow,” *arXiv:1802.05799*, 2018.

- [11] NVIDIA, “NCCL: The NVIDIA Collective Communications Library,” <https://developer.nvidia.com/nccl>, 2024.
- [12] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” *AISTATS*, 2017.
- [13] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates,” *ICML*, 2018.
- [14] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent,” *NeurIPS*, 2017.
- [15] H. Tang, S. Gan, C. Zhang, T. Zhang, and J. Liu, “Communication Compression for Decentralized Training,” *NeurIPS*, 2018.
- [16] A. Douillard, Q. Lhoest, A. Borzunov, et al., “DiLoCo: Distributed Low-Communication Training of Language Models,” *arXiv:2311.08105*, 2023.
- [17] Opentensor Foundation, “Bittensor: A Peer-to-Peer Intelligence Market,” Whitepaper, 2023.
- [18] Gensyn, “Gensyn Protocol: Trustless Training of Deep Neural Networks,” Whitepaper, 2023.
- [19] Cerebras, “SlimPajama: A 627B Token Cleaned and Deduplicated Dataset,” <https://huggingface.co/datasets/cerebras/SlimPajama-627B>, 2023.
- [20] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [21] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized Language Models,” *NeurIPS*, 2023.
- [22] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, and N. Lambert, “TRL: Transformer Reinforcement Learning,” <https://github.com/huggingface/trl>, 2022.
- [23] J. Benet, “IPFS – Content Addressed, Versioned, P2P File System,” *arXiv:1407.3561*, 2014.
- [24] Protocol Labs, “Filecoin: A Decentralized Storage Network,” Whitepaper, 2017.
- [25] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B. Yang, “High-Speed High-Security Signatures,” *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [26] Unsloth, “Unsloth: 2x Faster LLM Fine-Tuning,” <https://github.com/unslothai/unsloth>, 2024.
- [27] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” *arXiv:2307.08691*, 2023.
- [28] D. Kalajdzievski, “A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA,” *arXiv:2312.03732*, 2024.
- [29] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct Preference Optimization: Your Language Model Is Secretly a Reward Model,” *NeurIPS*, 2023.
- [30] V. Buterin, “A Next-Generation Smart Contract and Decentralized Application Platform,” Ethereum Whitepaper, 2014.